CampusCompute - Team Contributions

OS Level 3 Mini-Project (UE25CS242A) · Nov 7, 2026

Team

CampusCompute was built by two members, who split the work into Part A and Part B.

Role	Name	SRN	Responsibility
Person A	Adithya Jayanth	PES1UG25CS023	Part A: job scheduling and Banker's algorithm
Person B	Achyutha N Kumar	PES1UG25CS020	Part B: logger, IPC, workers, resource limits, cleanup

Work division

The project is split along its two halves: Person A owns scheduling and resource safety, Person B owns concurrency, processes and system resources. Both halves code against one shared header, `campuscompute.h`.

	Part A (Person A)	Part B (Person B)
Theme	CPU scheduling and deadlock avoidance	Threads, IPC, processes, limits, cleanup
Requirements	FR1, FR2, FR5, FR8	FR3, FR4, FR6, FR7
Source files	`scheduler.c`, `banker.c`, `main.c`	`logger.c`, `ipc.c`, `worker.c`, `limits.c`, `cleanup.c`, `integrate.c`, `partb.h`
Test target	`make test`	`make testb`

Person A: Adithya Jayanth (PES1UG25CS023)

Adithya built the scheduling simulator, the Banker's safety algorithm and the command-line interface.

Area	File	What was delivered
Job model and input (FR1)	`scheduler.c`, `campuscompute.h`	Job structure with status; job-file parser that range-checks every field, rejects duplicate IDs, and reports errors as `file:line: reason`
CPU scheduling (FR2)	`scheduler.c`	FCFS, non-preemptive Priority and Round Robin with a configurable quantum; fixed tie-breaking rules; IDLE gaps in the timeline
Reporting (FR2)	`scheduler.c`	Execution order, timeline, Gantt chart, per-job start, finish, turnaround and waiting times, averages, makespan and CPU utilisation
Policy comparison (FR2)	`scheduler.c`	`compare` command runs all three policies on one job set and names the lowest average waiting and turnaround time
Banker's algorithm (FR5)	`banker.c`	Need = Maximum − Allocation, step-by-step safety trace, safe sequence or blocked processes; built-in safe and unsafe cases plus a state-file loader with validation
Command-line interface (FR8)	`main.c`	Command dispatch, argument checking, usage text, clear error messages and exit codes
Test inputs and checks	`samples/`, `make test`	Job files (normal, idle gap, malformed) and Banker's state files; Part A self-test with positive and negative cases

Person B: Achyutha N Kumar (PES1UG25CS020)

Achyutha built the concurrency, inter-process and system-resource features, and the integrated demonstration that ties everything together.

Area	File	What was delivered
Concurrent logger (FR3)	`logger.c`	Producer-consumer with 3 producer threads, 1 consumer and a 5-entry circular buffer; one mutex with `not_full` and `not_empty` condition variables; clean shutdown with no lost events; writes `campuscompute.log`
Shared-memory IPC (FR4)	`ipc.c`	Parent and child exchange CPU-burst data through POSIX shared memory, synchronised by two named semaphores with 5-second timeouts
Worker failure detection (FR6)	`worker.c`	Three workers created with `fork()` and collected with `waitpid()`; exit codes and signals decoded; simulated failures (exit 42, SIGKILL) clearly labelled; zombie check
Resource limits (FR7)	`limits.c`	`setrlimit()` demos run in child processes: `RLIMIT_AS` makes a 256 MB allocation fail, `RLIMIT_CPU` stops a busy loop with SIGXCPU
Cleanup and verification (FR7)	`cleanup.c`	Removes only the program's own IPC objects, then verifies shared memory, semaphores, child processes and threads are gone
Integrated demo	`integrate.c`	`all` command runs every stage and prints a PASS/FAIL summary table (7 of 7 stages)
Shared internals and checks	`partb.h`, `make testb`	Object names, helpers and the Part B self-test, including bad-usage cases

Shared work

Some items belonged to both members rather than to one half.

- **Shared interface:** `campuscompute.h` defines the job model, input limits and the function prototypes both parts code against, so the two halves link into one program.
- **Build system:** the `Makefile` builds every source file into one `campuscompute` executable and provides the `make test` and `make testb` targets.
- **Documentation:** `README.md` covers usage, file formats, scheduling rules, Part B design notes and the table of resources and how they are released.
- **Testing and evidence:** both members ran their commands on Ubuntu, captured screenshots S01 to S24 with the SRN prompt, and checked the final submission zip.

Evidence in the screenshots

Every screenshot in the submission traces back to one member's work or to shared setup.

Screenshots	What they show	Owner
S01 to S05	Environment, compiler, project files, clean build, help text	Shared (S05 help text: Person A)
S06 to S09	FCFS, Round Robin, Priority and the three-way comparison	Person A
S10, S11	Producer-consumer logger: full-queue waits, clean thread shutdown	Person B
S12, S13	Shared-memory IPC with semaphores; IPC objects in `/dev/shm` removed by cleanup	Person B
S14, S15	Banker's algorithm: safe and unsafe states	Person A
S16, S17	Workers: normal run and simulated failures	Person B
S18	`setrlimit()` memory and CPU limits	Person B
S19	Unknown command and malformed job file produce clear errors	Person A
S20	Cleanup verification: no IPC objects, processes or extra threads left	Person B

Screenshots	What they show	Owner
S21, S22	Integrated run (7 of 7 stages pass) and the generated log file	Person B
S23, S24	Final files and the submission zip with integrity check	Shared